You can specify a multiline string by putting a pipe character (|) on the line with the attribute name. The multiline string can then be written over the next set of lines; each line must be indented relative to the line with the attribute name. Once again, outdenting indicates the end of the string:

```
runway:
  name: Project Runway
  due_date: 2016-12-18
  description: |
    The awesomest project ever.
    It's really, really great.
```

The Rails fixture-creation process uses information in your database to coerce the values to the proper type. I write dates in SQL format (yyyy-mm-dd), though any format readable by Ruby's Date.parse() will work.

The identifier that introduces each record is then used to access the individual fixture entry within your tests. Assuming that this is the Project class, you'd be able to retrieve these entries throughout your test suite as projects(:runway) and projects(:book), respectively. Unless you like trying to figure out what's special about projects(:project_10), I recommend meaningful entry names, especially for entries that expose special cases, like this: projects(:project_with_no_due_date).

The YAML data is converted to a database record directly in SQL, without using ActiveRecord#new or ActiveRecord#create. (To be clear, when you use the data in your tests, the objects you get are ActiveRecord models—ActiveRecord is bypassed only when the fixture is first converted to a database record.) This means you can't use arbitrary methods of the model as attributes in the fixture the way you can in a create call.

Fixture attributes have to be either actual database columns or ActiveRecord associations explicitly defined in the model. They can not be arbitrary setter methods defined in the model's code. Removing a database column from your model and forgetting to take it out of the fixtures results in your test suite erroring out when loaded. The fixture-loading mechanism also bypasses any validations you've created on your ActiveRecord, meaning there is no way to guarantee the validity of fixture data on load, short of explicitly testing each fixture yourself.

You do not need to specify the id for a fixture (although you can if you want). If you do not specify an id explicitly, the id is generated for you based on the entry's YAML identifier name. If you allow Rails to generate these ids, you get a side benefit: an easy way of specifying relationships between fixture objects. If your two model classes have an explicitly defined ActiveRecord relationship, you can use the fixture identifier of one object to define the relationship in